Gradient Descent Approach to IK

Summary:

This lesson introduces iterative (numerical) inverse kinematics. Instead of solving for the angles with triangles as in Lesson 6, students improve a guess: use forward kinematics to find where the tip lands, measure the error, and nudge every joint downhill. On Day 1, students apply the idea by hand to one number, a single joint and the simplified 2-link planar arm. On Day 2, students run the same loop in Python for the 3D arm using Lesson 4's DH forward kinematics, tune the learning rate, and explore what happens with different starting poses, out-of-reach targets and targets near full reach. By the end of the lesson, students can run gradient descent IK, check the result with FK, and compare it with the trigonometric method.

Objective:

By the end of this lesson, students will be able to:

- Describe iterative IK as guess, measure the error, improve, and explain why it needs only forward kinematics
- Write the cost as the squared distance from the tip to the target, and estimate a slope by nudging a joint
- Apply the update rule $\theta_{new} = \theta - \eta \times \text{slope}$ by hand to one number, a single joint and the 2-link planar arm
- Run gradient descent IK in code for the 3D arm and choose a sensible learning rate, tolerance and step limit
- Explain what happens when the target is out of reach, near full reach or approached from a different start pose, and always check the final error
- Compare gradient descent with the trigonometric method of Lesson 6

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-3 (evaluate solutions against trade-offs), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data
- Crosscutting Concepts: Systems and System Models; Stability and Change

Vocabulary:

Iterative (numerical) IK: Solving IK by improving a guess again and again instead of using a formula. It needs only forward kinematics.

Error: The distance from the end effector to the target.

Cost: A score for a guess: the squared distance from the tip to the target, measured in link lengths. The best angles make the cost as small as possible.

Slope (gradient): How much the cost changes when a joint turns a little. With several joints, the gradient is one slope per joint.

Gradient descent: Repeatedly moving each angle a small step in the direction that lowers the cost: $\theta_{new} = \theta - \eta \times \text{slope}$.

Learning rate (η): The size of each step. Too small is slow; too big overshoots and bounces.

Nudging: Estimating a slope by turning a joint by a tiny amount h and seeing how the cost changes: $(C(\theta + h) - C(\theta)) \div h$. It needs no calculus.

Tolerance: How small the error must be before we stop (this lesson: 0.5 mm).

Start pose: The joint angles we begin from. Gradient descent goes to the nearest valley, so the start can change which answer we get.

Radian: An angle unit where 180° = π ≈ 3.1416 rad (1 rad ≈ 57.3°). The update rule is used in radians.

Resources & Materials:

- Lesson 7 slideshow, Days 1 and 2 (33 slides; the slide numbers in the Procedures refer to it)
- Calculators with sin and cos keys (degrees and radians modes) for the Try It Yourself problems
- The assembled 4-DOF arm from Lesson 2, to show reach and the fully straight pose (optional)
- Computers with Python and NumPy, plus the helper files gd_ik.py (code in the Appendix) and dh_helper.py (from Lesson 4), for the Day 2 code slides

Procedures:

Day 1: Gradient Descent on the Flat Arm

- **Opening (4 min)** -- Display the title slide, then "Where Lesson 6 Left Off" and "Guess, Check, Improve" (slides 1–3). Trigonometry gave B = S = E = 45° exactly, but only for this arm; every new joint needs new triangles. Today the robot guesses, uses FK to see where the tip lands, measures the error, and nudges the angles. Play "hot and cold" with a volunteer to motivate the loop.
- **Measuring the error (4 min)** -- Show "Measuring the Error" (slide 4). The cost is the squared distance to the target, in link lengths. Squared means never negative, zero only on the target, and smooth. The goal is to make C as small as possible.
- **Rolling downhill (5 min)** -- Show "Rolling Downhill" (slide 5). The cost against an angle looks like a valley. Measure the slope, step the opposite way: $\theta_{new} = \theta - \eta \times \text{slope}$. Introduce η as the learning rate.
- **Worked example: one number (6 min)** -- Work through $f(x) = (x - 3)^2$ on slide 6 (slope $2(x - 3)$, η = 0.1, start at 0): 0.6, 1.08, 1.464, 1.7712, 2.017. Point out that the steps shrink as the slope flattens.
- **Try It Yourself #1 (5 min)** -- Students take steps with η = 0.25 and η = 1 (slide 7). Reveal 1.5, 2.25 for (a), and 6, 0, 6, 0… for (b). A step that is too big bounces forever; keep this idea for Day 2.

- **A single joint (5 min)** -- Show "Gradient Descent on a Single Joint" (slide 8): a 250 mm link at angle θ with the target at (250, 0), cost 2 − 2 sin θ, slope −2 cos θ, η = 0.2. Fill in the table together (61.21°, 72.24°, 79.23°…). Teacher note: the update uses radians; students convert to degrees at the end.
- **Measuring the slope by nudging (4 min)** -- Show slide 9. Instead of calculus, nudge the angle by h = 0.01 and compute (C(θ + h) − C(θ)) ÷ h = −1.40 at 45°, close to the exact −1.414. This needs only the cost, which needs only FK, so it works for any arm.
- **Try It Yourself #2 (4 min)** -- Students take two steps from θ = 60° (slide 10). Reveal 71.46° and 78.75°.
- **The flat arm (4 min)** -- Show "The Flat Arm: Two Joints" (slide 11). With S from vertical, X = L sin S + L sin(S + E) and Y = L cos S + L cos(S + E). Check S = E = 45° gives (426.78, 176.78). The gradient is now two slopes, one for S and one for E.
- **Worked example: (250, 250) (14 min)** -- Slides 12–16. Start at S = E = 0 (tip (0, 500)), η = 0.1, C = 2. Nudging gives slopes ≈ −4 (S) and −2 (E). The update gives S = 0.4 rad (22.92°) and E = 0.2 rad (11.46°), and the error falls from 353.6 mm to 187.0 mm. Show the iteration table (slide 15): it reaches 0.5 mm after 106 steps at S ≈ 0.1°, E ≈ 89.9°, matching Lesson 6 (S = 0°, E = 90°). Use slide 16 to compare: exact formulas vs. a general loop that needs only FK.
- **Try It Yourself #3 (3 min)** -- Students take one step for the target (341.51, 341.51) with slopes (−5.46, −2.73) (slide 17). Reveal S ≈ 31.3°, E ≈ 15.6°.
- **Recap & wrap-up (2 min)** -- Show the "Day 1 Recap" slide (slide 18): cost, slope, step, repeat. Preview Day 2: a computer does the nudging for all three joints.

Day 2: The 3D Arm in Code

- **Recap and extending to 3D (3 min)** -- Show "Extending to 3D" (slide 19). Nothing in the method changes: FK from Lesson 4 gives the tip, the cost is the squared distance, the gradient is three slopes (B, S, E), and we step all three downhill.
- **The code (5 min)** -- Show "Let the Computer Nudge" (slide 20): cost, gradient (nudge each joint up and down by h and divide by 2h) and the loop. Students open gd_ik.py (Appendix) and find these parts. The helper does the arithmetic.
- **Check it with code (4 min)** -- Run slide 21: ik_gd((301.78, 301.78, 176.78)) prints [45.0, 45.15, 44.7] 180 0.49, and FK prints the tip within 0.5 mm of the target. The angles are within 0.2° of Lesson 6's (45°, 45°, 45°).
- **Watching the error fall (4 min)** -- Show slide 22. The error drops from 449 mm to under 20 mm in about 10 steps, then crawls. Explain that the valley gets flat near the bottom, so the slope and the steps become tiny.
- **Choosing the learning rate (6 min)** -- Show slide 23: η = 0.02 needs about 900 steps, 0.05 about 360, 0.1 about 180, 0.2 about 85, and 0.3 never settles. Let students change eta in the code and watch. Use η = 0.1 as the safe default.
- **When do we stop? (3 min)** -- Show slide 24: stop when the error is under the tolerance or after a step limit. Hammer the point that stopping at the step limit does not mean the answer is right, so always check the final error.
- **Try It Yourself #4 (6 min)** -- Students run ik_gd((187.5, 324.76, 216.51)) (slide 25). Reveal [60.0, 30.12, 59.77] in 148 steps, close to Lesson 6's (60°, 30°, 60°). Challenge: try η = 0.2.

- **Different start, different answer (5 min)** -- Show slide 26. For (−324.76, 187.5, 216.51), start (10, 10, 10) finishes at B = −30°, S = −30°, E = −59.77° (the arm leans back), while start (100, 10, 10) finishes at B = 150°, S = 30.12°, E = 59.77°. Both put the tip on the target. Gradient descent goes to the nearest valley, so choose a start near the pose you want and check joint limits.
- **Try It Yourself #5 (7 min)** -- Students run ik_gd((−125, 216.51, 250), start=(100, 10, 10)) (slide 27). Reveal [120.0, 0.1, 89.84] in 100 steps, matching Lesson 4's (120°, 0°, 90°). Then try start (10, 10, 10) and discuss which way B turns. If time is short, assign this problem as homework.
- **Out of reach (4 min)** -- Show slide 28. For (400, 300, 200), trigonometry raised an error; gradient descent ends with the arm straight and pointing at the target, 38.5 mm short. Check the error to know the target was not reached.
- **When gradient descent struggles (3 min)** -- Show the table on slide 29: out of reach, near full reach (very slow, because the arm is almost straight and the slope is tiny), a bad η, and a different start.
- **Always check with FK (2 min)** -- Show slide 30. The loop already uses FK, so the final error is the check. If it is under the tolerance, send the angles to the motors; otherwise do not move.
- **Trigonometry vs. gradient descent (4 min)** -- Show slide 31. Ask: if the arm had six joints, which method would you use, and why? Gradient descent: the code barely changes, only FK grows. Trigonometry is exact and fast but needs new triangles for every joint.
- **Making it faster (2 min)** -- Show slide 32: warm starts from the current pose, Jacobian methods (exact slopes from calculus) and adaptive steps. All are still guess, measure, improve. If the lesson runs long, mention it in one sentence.
- **Wrap-up (2 min)** -- Show "What We Learned" (slide 33): iterative IK needs only FK; cost, slope and the update rule; learning rate and start pose matter; always check the final error. Preview Lesson 8.

Assessment:

Complete the Try It Yourself problems in the slideshow (#1–#5). Students check each answer against the solution on the slide and can verify #4 and #5 with the code.

Use these questions as an exit ticket or short quiz. The answers are in red.

1. For $f(x) = (x - 3)^2$ with slope $2(x - 3)$, start at $x = 0$ with $\eta = 0.25$. What is x after one step?
   Answer: $x = 0 - 0.25 \times (-6) = 1.5$.
2. Why does the cost use the squared distance instead of just the distance?
   Answer: It is never negative, it is zero only when the tip is on the target, and it makes a smooth bowl to roll down.
3. A single link has cost $C = 2 - 2 \sin \theta$ and slope $-2 \cos \theta$. Start at $\theta = 60°$ with $\eta = 0.2$. What is θ after one step?
   Answer: slope = −1, so $\theta = 1.0472 + 0.2 = 1.2472$ rad = 71.46°.
4. What does the learning rate η control? What goes wrong if it is too big, and too small?
   Answer: The size of each step. Too big: it overshoots and bounces without settling. Too small: it takes very many steps.
5. How can a computer find the slope of the cost without calculus?
   Answer: Nudge the joint a tiny amount h and see how the cost changes: $(C(\theta + h) - C(\theta)) \div h$. This

only needs FK.

6. True or False: if gradient descent stops, the answer is correct.
   Answer: False. It may have hit the step limit, or the target may be out of reach. Always check the final error.
7. The target (400, 300, 200) is out of reach. What does trigonometry do, and what does gradient descent do?
   Answer: Trigonometry detects $|\cos E| > 1$ and raises an error. Gradient descent ends at the closest point (arm straight, pointing at the target) with a leftover error of 38.5 mm.
8. Two different starting poses gave two different answers for the same target. Can both be right? Why?
   Answer: Yes. Both put the tip on the target. Gradient descent goes to the nearest valley, so the start pose decides which valid solution we get.
9. In one or two sentences, why might an engineer choose gradient descent over trigonometry for a six-joint arm?
   Answer: It is the same loop for any arm: only the FK function changes, while trigonometry needs new triangles and special cases for every joint. It costs speed and exactness.

Appendix: gd_ik.py

Save this code as gd_ik.py in the same folder as dh_helper.py. The code slides import ik_gd and tip from it.

```
"""Helper functions for Lesson 7: gradient descent IK for the 3-joint arm.

Angles are in degrees for you and radians inside the maths. Lengths are in mm.
Conventions (same as Lessons 4 and 6): z points up, B turns about z, S is measured
clockwise from vertical, and E is the clockwise bend of the elbow (0 = straight).
"""
import numpy as np
from dh_helper import dh_matrix

L = 250  # both links, in mm


def tip(angles):
    """Forward kinematics from Lesson 4. angles = (B, S, E) in radians -> (x, y, z)."""
    B, S, E = np.degrees(angles)
    T = dh_matrix(B, 0, 0, 90) @ dh_matrix(90 - S, 0, L, 0) @ dh_matrix(-E, 0, L, 0)
    return T[:3, 3]


def cost(angles, target):
    """Squared distance from the tip to the target, measured in link lengths."""
    e = (tip(angles) - target) / L
    return float(e @ e)


def gradient(angles, target, h=1e-4):
    """The slope of the cost for each joint, found by nudging that joint up and down by
h."""
    g = np.zeros(3)
    for i in range(3):
        nudge = np.zeros(3)
        nudge[i] = h
        g[i] = (cost(angles + nudge, target) - cost(angles - nudge, target)) / (2 * h)
    return g


def ik_gd(target, start=(10, 10, 10), eta=0.1, tol=0.5, max_steps=1000):
    """Gradient descent IK. Returns (B, S, E) in degrees, the steps used and the
leftover error in mm."""
    target = np.array(target, dtype=float)
    angles = np.radians(start).astype(float)
    for step in range(max_steps):
        error = np.sqrt(cost(angles, target)) * L
        if error < tol:
            break
        angles = angles - eta * gradient(angles, target)  # one step downhill
    error = np.sqrt(cost(angles, target)) * L
    return [float(v) for v in np.degrees(angles)], step, float(error)
```

```
if __name__ == "__main__":
    # The Lesson 7 worked example: the tip from Lesson 4.
    angles, steps, error = ik_gd((301.78, 301.78, 176.78))
    print([round(v, 2) + 0.0 for v in angles], steps, round(error, 2))  # [45.0, 45.15,
44.7] 180 0.49
```